ENTERPRISE ARCHITECTURE & USER MANUAL

InfraPilot Management Platform

Self-Service Infrastructure Provisioning, Governance, & Lifecycle Automation

1. Executive Summary & Purpose of the Application

InfraPilot is a production-grade, self-service enterprise infrastructure portal designed to eliminate the classic tension between developer agility and IT operational governance. In traditional enterprise environments, software engineering teams often endure multi-day delays waiting for infrastructure teams to manually approve and provision virtual machines (VMs) through ticketing systems. Conversely, granting developers unfettered direct access to cloud provider consoles creates severe security risks, untracked configuration drift, and runaway costs.

InfraPilot bridges this gap by providing an intuitive, web-based self-service catalog where developers can request compute infrastructure with automated quota checks. Behind the scenes, requests flow through an enterprise approval workflow, trigger asynchronous background job queues, execute HashiCorp Terraform automation scripts in isolated sandboxes, and catalog provisioned virtual machines in an immutable inventory with real-time power controls.

■ ***CORE VALUE PROPOSITION:*** *InfraPilot empowers engineers to provision infrastructure on-demand while guaranteeing IT leaders 100% policy compliance, strict budget quota enforcement, zero double-approvals, and complete audit logging.*

2. Key Capabilities & Architectural Highlights

- **Self-Service Catalog**: Standardized VM templates (OS distributions, CPU, RAM, Disk, Region, Environment) validated via Zod schemas.
- **Resource Quota Enforcement**: Real-time server-side validation preventing developers from exceeding allocated CPU, RAM, disk, or VM count limits.
- **Strict Separation of Duties (RBAC)**: Distinct roles: DEVELOPER (request, track, manage personal VMs) and ADMIN (fleet oversight, quota tuning, audit logs, approvals). Developers cannot approve requests; admins cannot approve their own requests.
- **Asynchronous Job Processing**: Dedicated BullMQ engine powered by Redis. HTTP REST requests return immediately while long-running Terraform automation executes reliably in background workers.
- **Terraform IaC Sandbox**: Executes terraform init, validate, plan, and apply in isolated directories using the safe hashicorp/local provider to generate JSON infrastructure manifests without cloud charges.
- **Fleet Inventory & Audit Trail**: Real-time state tracking with interactive Start/Stop power controls and an append-only compliance audit log recording all logins, approvals, and configuration changes.

3. System Architecture & How It Works

InfraPilot adopts a modern Decoupled Modular Monolith architecture, pairing a high-performance Express.js REST API with an autonomous BullMQ background worker and a React 18 single-page application (SPA).

Component	Tech Stack	Responsibility

Frontend Web Portal	React 18, Vite, Tailwind CSS, Recharts	Modern web dashboard, quick persona switching, real-time job status polling, interactive power controls, dynamic quota bars.
REST API Service	Node.js 22, Express.js, TypeScript, Zod, JWT	Handles authentication, RBAC authorization, request CRUD, approval state machine transitions, quota validation, and job enqueuing.
Database Layer	PostgreSQL 15, Prisma ORM	ACID-compliant relational store managing users, roles, quotas, requests, provisioning jobs, active VMs, and immutable audit logs.
Queue & Cache	Redis 7, BullMQ	In-memory reliable message broker managing the 'provisioning' queue with retry policies and worker coordination.
Provisioning Worker	Dedicated Node.js, Terraform CLI	Pulls jobs atomically from BullMQ, creates isolated execution sandboxes, runs Terraform stages (validate, plan, apply), captures logs, and updates DB.
Reverse Proxy	Nginx 1.27 Alpine	Serves compiled SPA assets on port 5173 and reverse-proxies /api/ requests to the API backend container.

4. Step-by-Step User Guide (How to Use InfraPilot)

InfraPilot includes built-in credentials with 1-click quick fill buttons on the login screen:

Persona	Email	Password	Access Level
Enterprise Admin	admin@infrapilot.local	AdminPass123!	Global Fleet Visibility, Approvals, Quotas, Audit Logs, Job Retries
Senior IT Developer	developer@infrapilot.local	DevPass123!	Self-Service VM Requests, Personal Inventory, Power State
DevOps Engineer	developer2@infrapilot.local	DevPass123!	Personal Inventory (Tenant Isolation test)

Developer User Journey

1. Open http://localhost:5173/login and click 'Developer Demo' -> 'Sign In'.
2. Note that personal metrics are displayed and admin-only menus are hidden.
3. Click 'Request VM' in the sidebar.
4. Quota Test: Select 16 Cores and 32GB RAM -> Red banner blocks submission exceeding maximum quota.
5. Valid Request: Select 2 Cores, 4GB RAM, 40GB Disk, VM Name: 'dev-api-node', Justification: 'API load test' -> Status PENDING_APPROVAL.
6. Power Actions: Navigate to 'VM Inventory' and click 'Stop' / 'Start' on active VMs to manage power state.

Administrator User Journey

1. Log out and click 'Admin Demo' -> 'Sign In'.
2. Fleet Dashboard shows global metrics across all developers and infrastructure.

3. Click 'Approvals': Review 'dev-api-node' at the top of the queue and click 'Approve'.
4. Click 'Provisioning Jobs': Observe real-time progress (VALIDATING -> PLANNING -> APPLYING -> COMPLETED).
5. Inspect Logs: Click 'Logs' to see live Terraform execution output and generated JSON manifests.
6. Adjust Quotas: Navigate to 'Resource Quotas' and adjust developer limits dynamically.
7. Review Audit Logs: Inspect the immutable trail of all actions with structured event payloads.

5. How to Run the Application

Single-Command Container Deployment (Docker Compose):
Run docker compose up -d from the project root.
• Web Portal: **http://localhost:5173**
• API Health Check: **http://localhost:5173/api/v1/health**
• Automated Test Suite: npm test (runs 13 Vitest verification tests)